



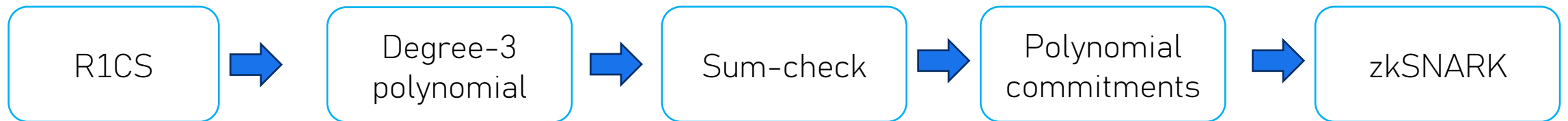
SPARTAN



NITHESH

Overview of spartan

- a family of zkSNARKs for proving the satisfiability of arbitrary R1CS instances without requiring a circuit-specific trusted setup.
- **Transparent:** no secret trapdoor is needed for preprocessing.
- **Succinct:** proof size and verifier cost are sublinear in the R1CS size.
- **Efficient prover:** prover cost ranges from $O(n)$ to $O(n \log n)$,



Spartan Core techniques

1. Compact R1CS Encoding

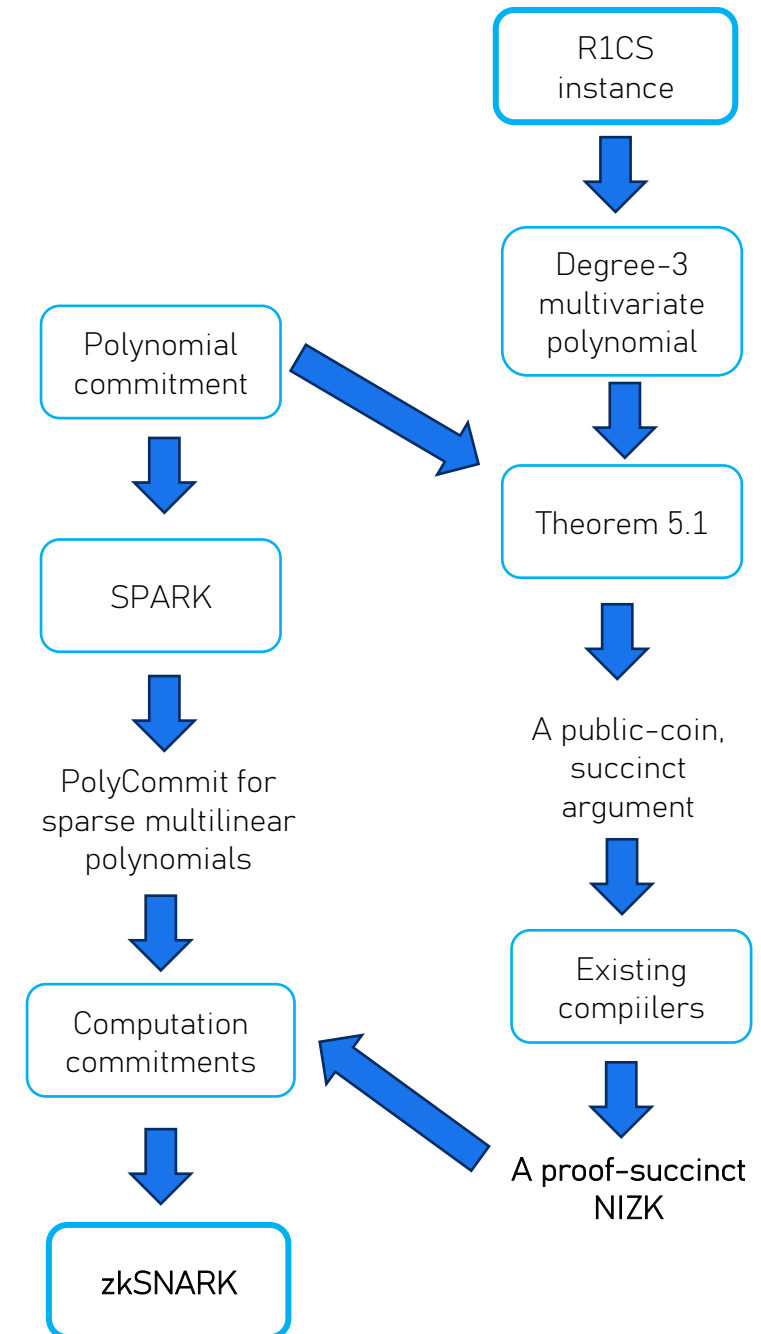
Encodes an R1CS instance as a degree-3 multivariate polynomial, decomposed into four multilinear polynomials for efficient sum-check.

2. Computation Commitments

Commit once to the R1CS structure, enabling later polynomial evaluations to be verified with sublinear verifier cost and no secret trapdoor.

3. SPARK

A compiler for efficiently committing to and proving evaluations of sparse multilinear polynomials, helping Spartan achieve a time-optimal prover.



R1CS (Recap)

- a way to represent an arithmetic circuit as algebraic constraints. Each constraint is written as:

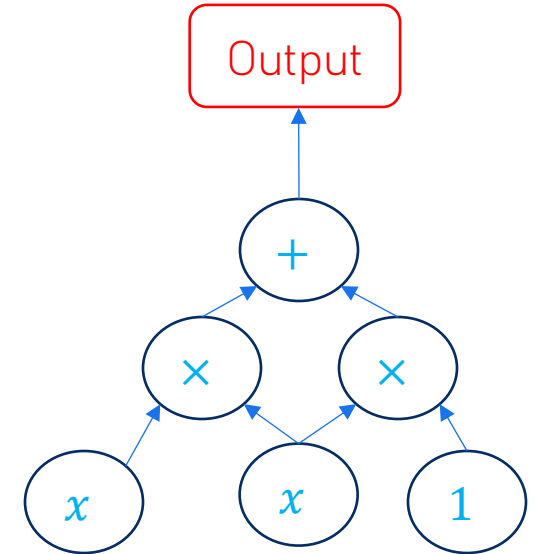
$$(A_q \cdot a) (B_q \cdot a) = C_q \cdot a$$

- $a = (a_0, a_1, \dots, a_m)$ is the vector of wire values, here $a_0 = 1$
- A_q, B_q, C_q coefficient vectors that define the q – th constraint
- Each constraint checks one multiplication relation

Matrix form used in Spartan:

$$z = (io, 1, w)$$
$$(Az) \circ (Bz) = Cz$$

- io : public input and output
- A, B, C : sparse $m \times m$ constraint matrices
- $\circ$: Hadamard product
- w : private witness



Example of y
 $= x^2 + x$

Multilinear extension (MLE)

Given a function $g: \{0,1\}^\ell \rightarrow \mathbb{F}$, its multilinear extension is the **unique multilinear polynomial**

$$\tilde{g}: \mathbb{F}^\ell \rightarrow \mathbb{F}$$

With degree at most 1 in each variable and satisfies

$$\tilde{g}(e) = g(e), \forall e \in \{0,1\}^\ell.$$

It can be computed as follows:

$$\tilde{g}(x) = \sum_{e \in \{0,1\}^\ell} g(e) \cdot \tilde{eq}(x, e),$$

$$\tilde{eq}(x, e) = \prod_{i=1}^{\ell} (x_i e_i + (1 - x_i)(1 - e_i)).$$

For Boolean inputs $x, e \in \{0,1\}^\ell$, $\tilde{eq}(x, e) = 1$ when $x = e$, and 0 otherwise. Hence, it selects the corresponding value $g(e)$ in the sum.

Encoding of R1CS instances

given R1CS instance $x = (\mathbb{F}, A, B, C, \text{io}, m, n)$, $s = \lceil \log m \rceil$

View the R1CS data as functions:

$$A, B, C: \{0,1\}^s \times \{0,1\}^s \rightarrow \mathbb{F}$$

$$Z = (\text{io}, 1, w), Z: \{0,1\}^s \rightarrow \mathbb{F}$$

Define:

$$F_{\text{io}}(x) = \left(\sum_{y \in \{0,1\}^s} A(x, y) Z(y) \right) \left(\sum_{y \in \{0,1\}^s} B(x, y) Z(y) \right) - \sum_{y \in \{0,1\}^s} C(x, y) Z(y)$$

For each constraint index x , this computes

$$(AZ)(x) \circ (BZ)(x) - (CZ)(x).$$

So: $F_{\text{io}}(x) = 0$ if the x -th R1CS constraint is satisfied.

Encoding of R1CS instances

Unfortunately, F_{io} is a function rather than a polynomial, so it cannot be used directly in the sum-check protocol.

$$\tilde{F}_{\text{io}}(x) = \left(\sum_{y \in \{0,1\}^s} \tilde{A}(x, y) \tilde{Z}(y) \right) \left(\sum_{y \in \{0,1\}^s} \tilde{B}(x, y) \tilde{Z}(y) \right) - \sum_{y \in \{0,1\}^s} \tilde{C}(x, y) \tilde{Z}(y)$$

Checking $\sum_x \tilde{F}_{\text{io}}(x) = 0$ is not enough, because nonzero terms may cancel. Let Q_{io} be the multilinear extension of the evaluations of $\tilde{F}_{\text{io}}$ on $\{0,1\}^s$. Therefore, if any constraint is unsatisfied, Q_{io} is not the zero polynomial.

$$Q_{\text{io}}(t) = \sum_{x \in \{0,1\}^s} \tilde{F}_{\text{io}}(x) \cdot \widetilde{\text{eq}}(t, x)$$

Then choose random $\tau \in \mathbb{F}^s$ and check: $Q_{\text{io}}(\tau) = 0$

Final Spartan polynomial:

$$G_{\text{io}, \tau}(x) = \tilde{F}_{\text{io}}(x) \cdot \widetilde{\text{eq}}(\tau, x), \quad \sum_{x \in \{0,1\}^s} G_{\text{io}, \tau}(x) = 0$$

$G_{\text{io}, \tau}$ has degree at most 3 in each variable and is the polynomial used in the sum-check protocol.

Sum check Protocol(Recap)

Used to verify a large sum of a low-degree polynomial over the Boolean hypercube.

$$T = \sum_{x_1 \in \{0,1\}} \sum \dots \sum_{x_\mu \in \{0,1\}} G(x_1, x_2, \dots, x_\mu)$$

Direct computation takes 2^μ evaluations.

Round 1

The prover sends: $g_1(X_1) =$

$$\sum_{x_1 \in \{0,1\}} \sum \dots \sum_{x_\mu \in \{0,1\}} g(X_1, x_2, \dots, x_\mu)$$

The verifier checks:

$$g_1(0) + g_1(1) = T.$$

It then chooses a random

$r_1 \in \mathbb{F}$ and sends it to the prover

Round i

The prover sends: $g_i(X_i) =$

$$\sum_{x_{i+1} \in \{0,1\}} \sum \dots \sum_{x_\mu \in \{0,1\}} g(r_1, \dots, X_i, \dots, x_\mu)$$

The verifier checks:

$$g_i(0) + g_i(1) = g_{i-1}(r_{i-1}),$$

then sends a new random challenge

$r_i \in \mathbb{F}$.

Final check

After μ rounds, the original claim is reduced to:

$$g_\mu(r_\mu) = G(r_1, \dots, r_\mu)$$

Sum check protocol

From the previous encoding, satisfiability reduces to:

$$\sum_{x \in \{0,1\}^s} G_{\text{io},\tau}(x) = 0, s = \lceil \log m \rceil.$$

The first sum-check reduces this large claim to: $e_x = G_{\text{io},\tau}(r_x), r_x \in \mathbb{F}^s$.

Since

$$G_{\text{io},\tau}(r_x) = \tilde{F}_{\text{io}}(r_x) \cdot \widetilde{\text{eq}}(\tau, x)$$

the verifier only needs to determine $\tilde{F}_{\text{io}}(r_x)$. However,

$$\tilde{F}_{\text{io}}(r_x) = \left(\sum_{y \in \{0,1\}^s} \tilde{A}(r_x, y) \tilde{Z}(y) \right) \left(\sum_{y \in \{0,1\}^s} \tilde{B}(r_x, y) \tilde{Z}(y) \right) - \sum_{y \in \{0,1\}^s} \tilde{C}(r_x, y) \tilde{Z}(y)$$

Main problem: Sending every value of $\tilde{Z}(y)$ would reveal or communicate the entire witness

Sum check protocol 1

To solve the problem we define:

$$\bar{A}(r_x) = \sum_{y \in \{0,1\}^s} \tilde{A}(r_x, y) \tilde{Z}(y), \quad \bar{B}(r_x) = \sum_{y \in \{0,1\}^s} \tilde{B}(r_x, y) \tilde{Z}(y), \quad \bar{C}(r_x) = \sum_{y \in \{0,1\}^s} \tilde{C}(r_x, y) \tilde{Z}(y)$$

Then:

$$\tilde{F}_{\text{io}}(r_x) = \bar{A}(r_x) \bar{B}(r_x) - \bar{C}(r_x).$$

The prover sends three claimed values: v_A, v_B, v_C .

The verifier checks:

$$e_x \stackrel{?}{=} G_{\text{io}, \tau}(r_x) = \tilde{F}_{\text{io}}(r_x) \cdot \widetilde{\text{eq}}(\tau, r_x) = (v_A \cdot v_B - v_C) \cdot \widetilde{\text{eq}}(\tau, r_x).$$

But the verifier must still verify:

$$\bar{A}(r_x) \stackrel{?}{=} v_A, \quad \bar{B}(r_x) \stackrel{?}{=} v_B, \quad \bar{C}(r_x) \stackrel{?}{=} v_C.$$

Sum check protocol 2

Instead of running three separate sum-check protocols, the verifier samples:

$$r_A, r_B, r_C \leftarrow \mathbb{F}.$$

It combines the claimed values into:

$$C = r_A v_A + r_B v_B + r_C v_C.$$

Define:

$$L(r_x) = \sum_{y \in \{0,1\}^s} r_A \cdot \tilde{A}(r_x, y) \cdot \tilde{Z}(y) + r_B \cdot \tilde{B}(r_x, y) \cdot \tilde{Z}(y) + r_C \cdot \tilde{C}(r_x, y) \cdot \tilde{Z}(y) = \sum_{y \in \{0,1\}^s} M_{r_x}(y)$$

Then using a **second sum-check protocol**, the verifier checks:

$$C \stackrel{?}{=} \sum_{y \in \{0,1\}^s} M_{r_x}(y) \Rightarrow e_y \stackrel{?}{=} M_{r_x}(r_y), \quad r_y \in \mathbb{F}^s.$$

$$M_{r_x}(r_y) = \left(r_A \cdot \tilde{A}(r_x, r_y) + r_B \cdot \tilde{B}(r_x, r_y) + r_C \cdot \tilde{C}(r_x, r_y) \right) \cdot \tilde{Z}(r_y)$$

Final Check

After the second sum-check:

$$M_{r_x}(r_y) = (r_A \cdot \tilde{A}(r_x, r_y) + r_B \cdot \tilde{B}(r_x, r_y) + r_C \cdot \tilde{C}(r_x, r_y)) \cdot \tilde{Z}(r_y)$$

The only remaining witness-dependent value is: $\tilde{Z}(r_y)$.

Before the sum-check protocols, the prover commits to the witness MLE:

$$C_w \leftarrow \text{PC.Commit}(\tilde{w}).$$

The prover later opens the commitment at: $r_y[1..]$.

$$v = \tilde{w}(r_y[1..]).$$

The prover supplies an evaluation proof π , and the verifier checks:

$$\text{PC.Verify}(C_w, r_y, [1..], v, \pi) = 1$$

The verifier reconstructs:

$$\tilde{Z}(r_y) = (1 - r_y[0]) v + r_y[0] (\widetilde{io, 1})(r_y[1..]).$$

Finally, it checks: $e_y =? (r_A \cdot \tilde{A}(r_x, r_y) + r_B \cdot \tilde{B}(r_x, r_y) + r_C \cdot \tilde{C}(r_x, r_y)) \cdot \tilde{Z}(r_y)$.

Spark

R1CS matrices $A, B, C \in \mathbb{F}^{m \times m}$ are sparse, with at most n non-zero entries. Instead of storing all m^2 entries, represent a matrix $M \in \{A, B, C\}$ using: $(row[k], col[k], val[k]), 0 \leq k < n$

Sparse polynomial evaluation

$$\tilde{M}(r_x, r_y) = \sum_{\substack{(i,j) \in (\{0,1\}^s, \{0,1\}^s) \\ M(i,j) \neq 0}} M(i,j) \cdot \widetilde{eq}(i, r_x) \cdot \widetilde{eq}(j, r_y)$$

Since M has at most n non-zero entries, this sum contains at most n terms instead of m^2 .

Naive approach

Computing each equality term separately requires a circuit of size

$$O(n\mu), \quad \mu = 2 \log m.$$

When instantiated with Hyrax-PC, proving this circuit requires

$$\boxed{O(n \log n) \text{ prover time}}$$

Spark Optimization

Previously, the equality values had to be computed separately for every non-zero entry:

$$\widetilde{eq}(\text{row}[k], r_x), \quad \widetilde{eq}(\text{col}[k], r_y)$$

This causes extra overhead: $O(n \log n)$

SPARK's idea:

Precompute equality tables:

$$\widetilde{eq}(0, r_x), \dots, \widetilde{eq}(m-1, r_x), \quad \widetilde{eq}(0, r_y), \dots, \widetilde{eq}(m-1, r_y)$$

Offline memory checking verifies that the prover retrieved the correct equality value for every row and column index. Assuming the underlying polynomial commitment has linear prover time, SPARK removes the logarithmic overhead:

$$O(n \log n) \rightarrow O(n)$$

The cost therefore scales with the number of non-zero entries n , rather than all m^2 matrix entries.

Time complexity

Let:

n : number of non-zero entries in the R1CS matrices

m : matrix dimension, with $n = O(m)$

Core interactive protocol

Component	Cost
Prover for two sum-checks	$O(n)$
Verifier for two sum-checks	$O(\log m)$
Sum-check communication	$O(\log m)$
Public matrix evaluations without SPARK	$O(n)$ verifier time

With SPARK and Hyrax-PC

Public preprocessing: $O(n)$
Prover: $O(n)$
Verifier: $O(\sqrt{n})$
Proof size: $O(\sqrt{n})$